SpecEdge：用于交互式大语言模型的可扩展边缘辅助服务框架

Jinwoo Park
KAIST
jinwoo520528@kaist.ac.kr

Seunggeun Cho
KAIST
sgn.cho@kaist.ac.kr

Dongsu Han
KAIST
dhan.ee@kaist.ac.kr

摘要

大语言模型（ LLMs ）驱动了许多现代应用，但大规模部署它们仍然成本高且资源密集。现有以服务器为中心的系统忽略了边缘的消费级 GPU。我们提出了 SpecEdge，一种边缘辅助推理框架，它使用推测解码方案将 LLM 工作负载在边缘和服务器 GPU 之间分配，仅通过网络交换 token 输出。SpecEdge 采用主动式边缘草稿生成，将边缘 token 生成与服务器验证重叠，并通过流水线感知调度交错多个用户请求以提高服务器端吞吐量。实验表明，SpecEdge 通过实现服务器吞吐量提高 **2.22×**，整体成本效率提高 **1.91×**，相比仅服务器基线，token 间时延降低 **11.24%**，为 LLM 服务引入了一种可扩展、成本效益高的新范式。代码可在 https://github.com/kaist-ina/specedge 获取。

1 引言

大语言模型（ LLMs ）已经成为现代应用程序的核心部分，如对话式人工智能、代码生成和实时内容创作 [Dubey 等 , 2024, Lozhkov 等 , 2024, Achiam 等 , 2023, Touvron 等 , 2023, Jiang 等 , 2023, Brown 等 , 2020]。然而，在平衡运营成本与时延要求时，扩大 LLM 部署以满足不断增长的需求仍然具有挑战性。

通过在网络边缘利用消费级 GPU，有一个极具吸引力的机会可以显著降低大语言模型的服务成本。GeForce RTX 4090 [Nvidia, 2025a] 在 FP16 张量操作和 FP16 累加运算中可提供高达 330.3 TFLOPS 的性能 [Nvidia, 2024a], , 超过了数据中心级 A100 [Nvidia, 2025b], 的 312 TFLOPS 性能，其成本低 14.43 倍 [GCP, 2025, Vas, 2025]。随着这些强大边缘设备的广泛可用 [Valve, 2024, Nvidia, 2024b], , 一种将计算负载转移到这些性价比高的资源上的边缘辅助推理方法可能从根本上改变大语言模型部署的经济性。

尽管有这个机会，现有的推理架构未能有效利用这些边缘资源。现有的并行化技术 [Shoeybi 等人，2019，Rasley 等人，2020，Aminabadi 等人，2022] 在数据中心内部拆分计算的方法，在公共互联网环境下会失效，因为高时延和有限的带宽使得频繁传输中间结果变得不切实际。主流方法如张量并行和流水线并行依赖于高带宽、低时延的互连，如 NVLink 或 InfiniBand [Nvidia，2025c,d]。在广域网（ WAN ）中，在边缘与服务器 GPU 之间传输中间模型状态会迅速变得不可行，从而阻止这些异构资源之间进行有意义的协作。

在本文中，我们提出了 SpecEdge，这是第一个实际的边缘辅助推理框架，通过在消费级边缘 GPU 和云服务器之间分配计算，从根本上降低了大语言模型（ LLM ）的服务成本。SpecEdge 的核心创新在于其能够有效地在广域网中协调边缘和服务器资源 —— 这是传统方法以前无法实现的能力。

第 39 届神经信息处理系统大会（ NeurIPS 2025 ）。

并行化技术。为实现这一点，SpecEdge 采用了一种推测解码范式，将 LLM 推理分为边缘草稿生成和服务器验证，仅交换 token 输出而不是完整的模型状态。这种方法显著减少了带宽需求，同时最小化了通信轮次。

为了使这种边缘辅助范式在现实部署场景中切实可行，SpecEdge 实施了两项关键的启用技术。我们的 主动式边缘草稿生成 允许边缘 GPU 在等待服务器验证的同时继续生成 token，有效地通过本地计算掩盖网络和验证时延。与此互补，我们的 服务器端流水线感知调度通过智能批处理协调来自多个用户的验证请求，以保持 GPU 的高利用率。这些技术共同确保了边缘辅助推理固有的成本优势不会被网络限制或资源利用效率低下所削弱。

我们的评估验证了这种边缘辅助方法的有效性，显示 SpecEdge 实现了 **1.91×** 更高的成本效率，同时将服务器端吞吐量提高了 **2.22×** ，并将 token 间时延降低了 **11.24%**。即使在具有挑战性的广域网条件下，这些改进仍然存在，性能优于零网络时延的仅服务器基线。通过有效利用广泛可用的边缘 GPU，SpecEdge 为可扩展且经济高效的 LLM 服务确立了一种新的范式，以应对生成式 AI 应用日益增长的计算需求。

2 背景与相关工作

LLM 服务系统。现代 LLM 框架通过各种优化解决时延、吞吐量和资源效率的挑战。DeepSpeed-Inference [Rasley 等人，2020] 以及 TensorRT-LLM [Nvidia, 2024c] 利用低级 GPU 优化、模型并行和量化，而 vLLM [Kwon 等人，2023] 引入了 PagedAttention 以实现高效的内存管理。现代并行化策略 [Shoeybi 等人，2019，Aminabadi 等人，2022] 通过多 GPU 提升性能。然而，这些方法依赖于通过专用互连（ In niBand 、 NVLink ）连接的数据中心 GPU，其吞吐量超过数百 GB/s [Nvidia, 2025c]—— 这种速度在广域网（ WAN ）上无法实现。

推测解码。推测解码 [Leviathan 等，2023，Chen 等，2023] 通过让一个较小的辅助模型生成多个候选 token，以供主模型并行验证，从而降低时延。该过程包括三个阶段：候选 token 的草稿生成、验证以及生成 token 的调和。最近的进展集中在更高效的草稿生成方法，要么使用更轻量的辅助模型，要么使用参数减少的目标模型 [Bhendawade 等，2024，Cai 等，2024，Li 等，2024，Stewart 等，2024，Zhang 等，2023]。

基于树的推测解码。 标准推测解码在序列长度增加时会遭遇指数下降的接受率 [Leviathan 等 , 2023, Chen 等 , 2023]. 基于树的方法 [Miao 等 , 2024, Chen 等 , 2024, Svirschevski 等 , 2024, Cai 等 , 2024, Sun 等 , 2024] 通过同时探索多条路径来解决这一问题。Sequoia [Chen 等 , 2024] 以及 SpecExec [Svirschevski 等 , 2024] 通过剪枝不太可能成功的分支进一步优化。

分布式大语言模型服务。 像 Petals 这样的分拆推理方法 [Borzunov 等， 2024] 以及 Helix [Mei 等, 2024] 在多个设备上分布模型层和流水线请求，相对于内存卸载方法提高吞吐量 [Ren 等，2021, Pudipeddi 等， 2020], 但会引入网络时延，使得时延相比数据中心解决方案增加。若干工作 [Timor 等，2025，Liu 等，2025，McDanel， 2024] 在服务器节点内使用多个 GPU 设备来重叠推测解码的草稿和验证任务，从而实现更快的推理，但代价是更高，因为每个查询需要额外的服务器设备。

设备端大语言模型推理。 像 MLC LLM 这样的框架 [MLC 团队，2023-2024] 以及 Web LLM 实现了使用较小或量化模型的完全设备端推理。虽然近期的方法 [Svirschevski 等，2024，Song 等，2024，Xue 等，2024] 展示了利用用户设备的潜力，但它们会影响输出质量和时延。相比之下，我们的方法在服务器端保留了高质量的验证阶段 —— 实现了一种混合解决方案，其表现优于纯集中式或完全本地的推理。

3 问题与动机

为大语言模型（ LLMs ）提供服务由于其资源密集型特性而面临显著的计算挑战。虽然数据中心依赖昂贵的 H100 和 A100 GPU，但边缘存在丰富的计算资源，以消费级 GPU 的形式存在。如图 1 所示，边缘设备如 RTX 4090 和 RTX 3090 在运行像 Qwen2-0.5B 这样的小型但功能强大的语言模型时，每生成一个 token 的成本比服务器级 GPU 低大约 **30-50×** 。尽管具有这种成本优势且广泛可用，但当前的 LLM 服务架构未能整合这些边缘资源，从而错过了通过分布式推理在降低成本的同时保持高质量 LLM 服务的重大机会。

传统分割计算。 一种利用边缘 GPU 的方法是在边缘和服务器资源之间进行分割计算。分割计算在机器学习文献中已经被广泛研究 [Kang 等 , 2017, Zhou 等 , 2019, Wang 等 , 2020]，早于变换器架构。传统上，这种方法将神经网络层在服务器和边缘之间划分，通过传输中间张量而非原始输入数据来降低服务器的计算负载或最小化网络通信。然而，由于变换器的独特特性，这种方法并不适合 LLM 服务 [Vaswani, 2017]:

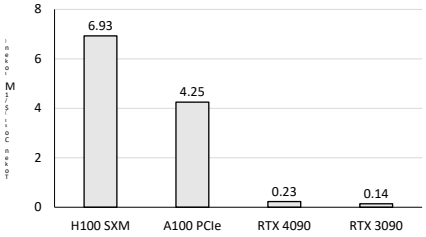


图 1: 与 Qwen2-0.5B 模型的 token 生成成本比较。

限制 #1：过高的时延。虽然层级拆分已经在分布式点对点系统中使用 [Borzunov 等 , 2024, Mei 等 , 2024] 以支持在普通 GPU 上无法实现的大模型，这些方法更注重可行性而非性能。这是因为，与早期的机器学习应用不同 [He 等 , 2016, Redmon, 2016, Diwan 等 , 2023] 早期应用通常只涉及少数通信轮次，而由于自回归特性，LLM 应用每生成一个 token 就需要通信。在我们的分布式网络设置中（ §5.3 ），相较于仅服务器解决方案，层级拆分将时延增加了 2.35 倍。

限制 #2：在减少 I/O 方面效果有限。LLM 推理本质上受制于内存 I/O。[Pope 等人，2023]，指出，生成单个 token 需要访问整个模型的参数。Transformer 架构中，例如通用矩阵 - 向量乘法（ GEMV ）等操作，相对于其内存 I/O 需求算力强度较低。虽然逐层拆分计算通过将层卸载到边缘设备来减轻计算负载，但无法缓解推理的 I/O 受限特性。一个更有效的方法是批量验证多个候选 token，这显著降低每个 token 的 I/O 开销并提高 GPU 吞吐量。

4 SpecEdge 设计

我们的目标是将消费级边缘 GPU 与服务器资源结合起来，在不影响质量或时延的情况下实现成本高效的大语言模型服务。

4.1 分解型大语言模型解码

为了克服传统分割计算的限制，我们将 token 草稿生成移到边缘 GPU 上，同时在服务器上保持验证。与传统的大语言模型服务不同，传统服务中服务器处理整个过程，SpecEdge 将推测解码拆分为两个不同的阶段：基于边缘的草稿生成和基于服务器的验证。

在我们新的设计中，边缘 GPU 生成候选 token 并将其发送到服务器，服务器在一次前向传递中验证这些 token。服务器返回经过验证的 token 以及一个额外的 token 给边缘设备，边缘设备随后更新其序列、 KV 缓存，并继续生成。这一循环重复进行，直到生成序列结束 token。关键是，这种方法保持了服务器模型的精确输出分布 [Leviathan 等，2023]—— 即使在使用不同模型进行生成和验证时，最终输出也保证从与仅由服务器模型生成时相同的分布中采样。

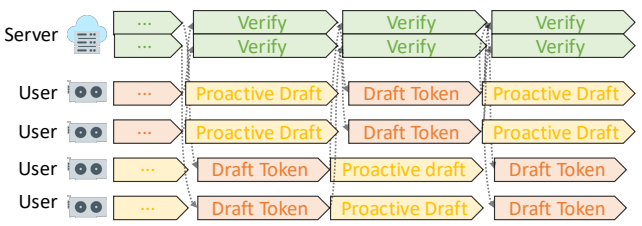


图 2：SpecEdge 的抽象时间线，包含草稿（边缘侧）和验证（服务器端）推理概念。

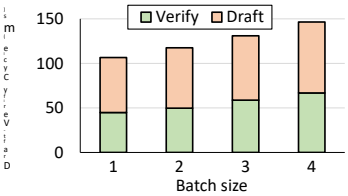


图 3：由草稿和验证阶段组成的推测解码循环。

如图 2 所示，这种拆分方法使服务器能够专注于验证，而边缘设备则处理自回归草稿生成过程。通过将大部分草稿生成时间转移到廉价的边缘 GPU，我们显著降低了整体服务成本，同时提高了服务器吞吐量。

图 3 量化了这种方法的潜在优势，显示了在不同批处理大小下，草稿阶段占据了草稿 - 验证循环的主要部分。通过将这一主要环节卸载到边缘设备，SpecEdge 可以显著减少服务器时间并提高整体系统效率。

This 分解方法相比传统的分布式计算承诺有两个基本优势 g:

- **解决过高时延问题（限制 #1）：**与需要逐 token 通信轮次的分层拆分不同，SpecEdge 通过一种推测解码方法大幅减少了网络交互。这既最小化了客户端与服务器的往返次数，也减少了传输数据的量，从而消除了传统拆分方法固有的过高时延。
- **解决 I/O 瓶颈（限制 #2）：**通过在单次服务器端前向传递中启用对多个 token 的批量验证，SpecEdge 在多个 token 验证之间分摊了模型参数访问的开销。这提高了每次操作的算术强度，直接应对了大语言模型推理的 I/O 受限特性，并显著提升了 GPU 吞吐量。

然而，要充分实现这些好处，我们必须克服在不同设备上分离草稿生成和验证时出现的两个关键挑战：

- **潜在时延增加：**在传统的推测解码中，草稿和验证在同一设备上顺序进行，转换开销最小。在我们的分离式环境中，天真地实现这一序列会在每个草稿 - 验证周期增加网络往返时延，可能抵消我们的时延优势并恶化用户体验。
- **服务器未充分利用的风险：**如果没有仔细协调，当边缘设备完成草稿制作时，服务器 GPU 将保持空闲。这种低效率可能会严重限制吞吐量，并削弱我们方法的成本效益，尤其是在为多个具有不同工作负载模式的并发用户服务时。

为了释放分解式推测解码的全部潜力，SpecEdge 引入了两项关键创新：主动边缘草稿生成 (§ 4.2)，通过不断生成 token 候选而不等待验证结果来掩盖网络时延，以及流水线感知调度 (§ 4.3)，通过智能地批处理来自多个用户的验证请求来最大化服务器 GPU 利用率。这些技术结合起来，使 SpecEdge 能够同时实现低时延和高成本效率。

4.2 边缘的主动草稿生成

我们的关键洞见是在服务器验证期间继续生成草稿，从而消除空闲时间。边缘 GPU 执行两类草稿生成：初始草稿生成用于产生第一批候选 token，主动式草稿生成则在服务器验证期间继续进行。当边缘 GPU 向服务器发送 n 个候选 token 后，它会立即继续生成额外 token，而不等待验证结果。若验证过程中有任何 token 被拒绝，这些主动生成的 token 就会被丢弃。不过，当全部 n 个 token 均被接受，且服务器的奖励 token 与主动生成的第一个 token 一致时，我们将这种情况称为 “ 完全草稿对齐 ”。此时，额外生成的 token 可立即使用，从而有效掩蔽网络和验证时延。通过使计算与通信重叠，这种方法显著降低了端到端时延，消除了草稿生成与验证周期之间的等待。同时，它还通过以下方式提高服务器吞吐量：

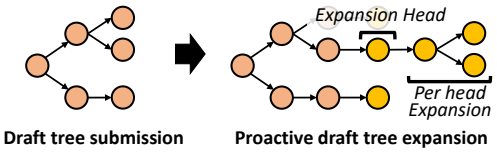


图 4：主动草稿树扩展的示例性例子。

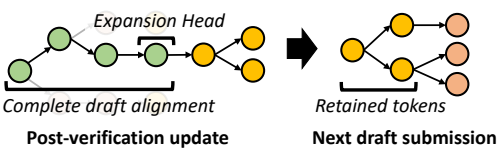


图 5：验证后更新，包括完整的草稿对齐以及随后的草稿提交。

确保验证请求的连续流动，使昂贵的服务器 GPU 保持高度利用，而不是在等待边缘设备完成草拟时闲置。

初始草稿生成。SpecEdge 采用最先进的基于树的草稿生成方法 [Svirschevski 等人，2024]。然而，SpecEdge 被设计为面向未来，不依赖于任何特定的草稿生成技术。它仅需要一个生成候选 token 的草稿生成阶段，并且兼容各种推测解码方法，包括基于树的方法 [Chen 等人，2024, Miao 等人，2024], 有损 / 无损方法，以及早期的单序列方案 [Leviathan 等人，2023, Chen 等人，2023]。随着该领域的发展，任何推测解码的进展都可以无缝集成。

主动草稿树扩展。与在生成初始 token 后就停止的传统推测解码不同，我们的方法持续主动地进行草稿，以预测未来的服务器响应。该主动扩展的预期收益可以表示为：

$$\mathbb{E}(\text{Gain}) = P_{\text{align}} \cdot P_{\text{match}\,|\,\text{align}} \cdot \left(\begin{matrix} T_{\text{draft}} \\ H_{\text{expan}} \end{matrix} - 1 \right) \tag{1}$$

其中 P_{align} 表示已验证序列与草稿生成序列之间对齐的概率，$P_{\text{match}\,|\,\text{align}}$ 是在对齐的情况下服务器的额外 token 与扩展头匹配的概率，T_{draft} 是主动草稿生成 token 的总数，H_{expan} 是扩展头的数量（从这些叶节点继续草稿生成）。

这种公式揭示了一个基本的权衡：增加 H_{expan} 会提高对齐概率（ P_{align} ），但会降低词元保留率（ $\frac{T_{\text{draft}}}{H_{\text{expan}}} - 1$ ），而较少的头会降低对齐概率，但在对齐成功时显著提高保留率。天真地应用初始草稿生成策略 —— 将每个叶节点视为扩展头 —— 虽然最大化了 P_{align}，但收益微乎其微。我们的实证结果表明了一种违反直觉却最优的方法：在生成初始草稿树之后，SpecEdge 会识别累计对数概率最高的单一路径，并仅从该节点继续草稿生成（图 4 ）。这种集中策略通过在对齐发生时产生显著更高的收益来最大化预期收益，尽管对齐概率降低。

验证后更新。当服务器验证完成后，边缘 GPU 会收到已接受的 token 以及在服务器前向传播过程中生成的一个额外 token。此时，SpecEdge 会将服务器的输出与其主动生成的 token 进行比较。此外，边缘会根据已接受的 token 更新其草稿模型 KV 缓存。

如果服务器接受的 token 和额外 token 完美匹配主动草稿生成树中的一条路径 —— 一个完整的草稿对齐 ——SpecEdge 会保留该分支，并允许该边从这个高级位置继续草稿生成（图 5 ）。这种方法消除了重新生成已草稿生成 token 的需要，通过高效地重用先前的计算，为下一轮草稿生成生成更深入的草稿。

如果完全对齐失败 —— 无论是因为验证路径偏离了前瞻树，还是没有到达叶节点 —— SpecEdge 都会丢弃前瞻工作并恢复到初始草稿，从验证序列的末端重新构建树。虽然这种情况无法从前瞻草稿中受益，但该策略仍然通过利用成功对齐时的结果来提高平均性能。

4.3 服务器端流水线感知调度

与传统的推测解码不同，传统方法中服务器既进行草稿生成又进行验证，SpecEdge 则将服务器的 GPU 完全用于验证。这种分离使服务器能够专注于基于批次的验证，而不是逐 token 生成，从而显著提高

资源利用。然而，这种分解的方法也带来了新的调度挑战：当边缘正在草稿生成下一组 token 时，如果服务器仅等待同一请求的验证任务，它将处于空闲状态。这种模式会在系统的一部分等待另一部分时产生未利用计算能力的 “ 空档 ” 。

管线感知的验证调度。SpecEdge 通过在不同边缘设备上处理的多个请求中交错执行验证任务，消除了计算效率低下的问题。服务器持续验证来自一组请求的已完成草稿批次，同时其他请求在各自的边缘设备上生成新的草稿。这种流水线方法确保了对传入草稿的即时处理，验证后的请求会迅速返回其边缘设备进行额外的草稿生成，从而释放服务器资源用于下一批验证。通过将边缘设备数量与服务器验证能力对齐，这种编排相比相同批量的传统仅服务器配置有效地将服务器吞吐量提高一倍，从而显著提升了成本效率和 GPU 利用率。

为了实现最佳的流水线效率，SpecEdge 使用实时性能测量动态校准服务器验证时间与边缘操作之间的关系。系统会调整草稿深度 —— 即通过草稿模型的前向传递次数 —— 以满足以下公式：服务器验证时间 ≈ 边缘草稿时间 + 网络往返时延。这种校准确保 token 批次在服务器完成验证前一个批次时正好到达，从而消除计算空洞，并在整个分布式系统中最大化资源利用的同时将端到端时延降至最低。

处理异构请求。服务器端验证必须高效地处理包含不同序列长度的请求批次 —— 这是当多个用户处于生成过程的不同阶段时的常见情况。SpecEdge 通过两种互补技术应对这一挑战。首先，它为批次中每个 token 序列采用自定义注意力掩码，确保模型仅关注该序列内的有效 token，同时在不干扰跨序列处理的情况下实现并行处理。其次，它实现了 KV 缓存填充，以匹配批次中最长的序列，避免在推理过程中重新配置张量形状的巨大计算开销，同时对较短序列产生的额外开销极小。

这种双重方法使 SpecEdge 能够以统一的批次处理多样化的验证请求，充分利用 GPU 并行性，同时适应边缘到服务器通信的异步特性。其结果是在不牺牲交互应用所需响应性的情况下，实现服务器吞吐量最大化。

5 评估

我们在边缘辅助服务器配置下对 SpecEdge 进行了评估，并将其与仅服务器配置在各种大语言模型和数据集上的表现进行了比较。我们的发现总结如下：

• SpecEdge 通过将服务器吞吐量提高，**与仅服务器环境相比，平均提高了 1. 效率**。**22×** 平均。

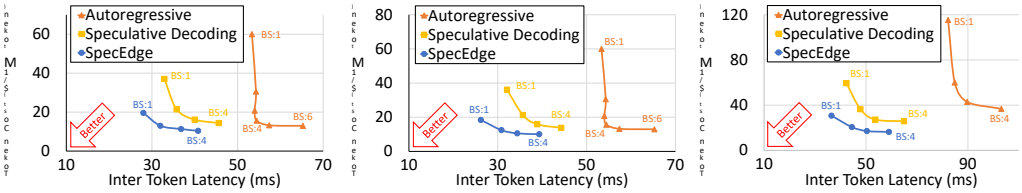
• 它将 token 间时延平均降低了 **11.24%**，即使在服务器与边缘之间存在 14.07ms 的往返时延，也优于没有网络时延的仅服务器配置。

实施与设置。我们系统的边缘辅助配置使用了一个服务器端的 NVIDIA A100 GPU，通过广域网连接到多个边缘端的 NVIDIA RTX4090 GPU。RTX4090 GPU 的数量会随着并发请求数（批量大小 x 2 ）而扩展。在我们的实验中，我们测得本地边缘节点与我们的 Google Cloud 实例之间的平均往返时延（ RTT ）为 14.07 毫秒。我们在不同的操作条件下，对各种模型和数据集进行了评估。代码可在 https://github.com/kaist-ina/specedge 获取。

基线和指标。我们的主要基线是仅服务器配置，采用基于树的推测解码，辅以自回归解码和分层拆分方法，将部分大语言模型的层卸载到边缘设备。SpecEdge 可以利用客户端或边缘 GPU ； 在本次评估中，我们假设它使用来自边缘云提供商的消费级 GPU。根据提供商定价 [GCP, 2025, Vas,2025]，服务器端 A100 40GB GPU 的费用为每小时 4.05 美元，而运行 32B 模型需要 A100 80GB，费用为每小时 5.05 美元。相比之下，SpecEdge 每使用一块 RTX 4090，额外增加 0.35 美元每小时。我们的关键指标包括成本效率

Table1：SpecEdge 与仅服务器方法之间的吞吐量和成本效率比较。

Target/Draft	Task	Gen. tokens per verify		Server Throughput (tok/s)		Cost Efficiency (1k toks/$)	
		Server-only	**SpecEdge**	Server-only	**SpecEdge**	Server-only	**SpecEdge**
Qwen3 14B/1.7B	Multi-turn bench	3.92±1.51	3.98±1.57	31.78	66.54 (**2.09x**)	28.25	50.60 (**1.79x**)
	Translation	3.95±1.47	4.25±1.45	32.24	65.25 (**2.02x**)	28.66	49.47 (**1.73x**)
	Summarization	3.73±1.60	3.95±1.61	29.70	67.53 (**2.27x**)	26.40	51.22 (**1.94x**)
	QA	3.42±1.57	3.59±1.56	27.30	62.04 (**2.27x**)	24.26	47.09 (**1.94x**)
	Math.	4.10±1.48	4.25±1.49	32.84	72.93 (**2.22x**)	29.19	55.28 (**1.89x**)
	RAG	3.73±1.53	3.83±1.56	29.89	64.04 (**2.14x**)	26.57	48.78 (**1.84x**)
Qwen3 14B/0.6B	Multi-turn bench	3.87±1.41	4.41±2.25	33.45	69.58 (**2.08x**)	29.73	52.97 (**1.78x**)
	Translation	3.79±1.48	4.67±2.34	32.88	69.00 (**2.10x**)	29.22	52.23 (**1.79x**)
	Summarization	3.68±1.49	4.21±2.16	31.17	68.60 (**2.20x**)	27.71	52.16 (**1.88x**)
	QA	3.33±1.41	3.79±1.94	28.89	61.90 (**2.14x**)	25.68	46.98 (**1.83x**)
	Math	3.90±1.57	5.27±2.27	33.53	83.88 (**2.50x**)	29.80	63.56 (**2.13x**)
	RAG	3.53±1.52	4.29±2.16	30.07	69.51 (**2.31x**)	26.73	52.76 (**1.97x**)
Qwen3 32B/1.7B	Multi-turn bench	4.22±1.93	4.71±2.66	24.96	56.47 (**2.26x**)	17.80	35.38 (**1.99x**)
	Translation	4.08±1.97	5.24±2.84	24.33	58.79 (**2.42x**)	17.34	36.83 (**2.12x**)
	Summarization	4.19±2.01	4.52±2.68	24.33	54.07 (**2.42x**)	17.65	33.90 (**2.12x**)
	QA	3.62±1.95	3.93±2.49	21.59	46.14 (**2.14x**)	15.39	28.99 (**1.88x**)
	Math.	4.60±1.93	5.40±2.78	27.52	64.01 (**2.33x**)	19.62	40.18 (**2.05x**)
	RAG	3.89±2.05	4.19±2.69	22.67	49.46 (**2.18x**)	16.16	31.05 (**1.92x**)



(a) 14B-1.7B (b) 14B-0.6B (c) 32B-1.7B

图 6：不同批量大小（ BS ）和模型组合下，服务器端基线与 SpecEdge 在每个 token 成本和 token 间时延的比较。

（每美元生成的 token 数）、服务器吞吐量（每单位时间生成的 token 数）和 token 间时延（用户感知的输出时延）。所有配置产生相同的输出分布，因为它们使用相同的基础模型。

模型和数据集。我们使用四种不同的大语言模型（ LLM ）：Qwen3-32B/14B [团队，2025], Vicuna-33B [Chiang 等人，2023] 以及 Llama2-13B-chat-hf [Touvron 等人，2023]。除非特别指出，否则所有模型的温度设置为 0.7。对于草稿模型（ draft models ），我们使用五种不同的模型：Qwen3-1.7B/0.6B [团队，2025], Sheared Llama-1.3B [Xia 等人，2023], Tiny Llama-1.1B [Zhang 等人，2024], 以及 JackFram-160M [Miao 等人，2024]。最后，我们使用 SpecBench [Xia 等人，2024], C4 (en) [Raffel 等人，2020], OpenAssistant 对话 数据集 [Köpf 等 , 2024],WikiText-2 [Merity 等 , 2016], 和 MTBench [Zheng 等 , 2023].

5.1 端到端性能与成本效益

表 1 比较了 SpecEdge 与仅服务器推测解码基线在六项 SpecBench 任务上的服务器吞吐量和成本效率。我们使用批大小 1 ；该设置在 SpecEdge 和仅服务器基线中均实现最低的 token 间时延，适合对时延敏感的交互式 LLM 服务。在端到端评测中，相比仅服务器配置，SpecEdge 通过 **2.22×** 的吞吐量提升，平均实现 **1.91×** 的成本效率提升。尽管使用消费级边缘 GPU 会略微增加成本，

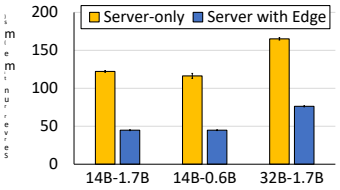


图 7：仅服务器与带边缘草稿生成服务器的运行时间比较。

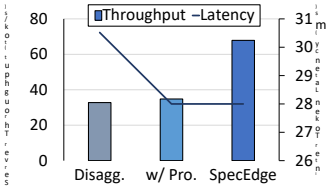


图 8：与 SpecEdge 组件消融的性能比较。

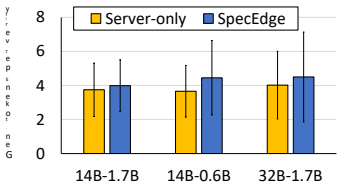


图 9：server-only 与 SpecEdge 之间每次验证生成的 token 数。

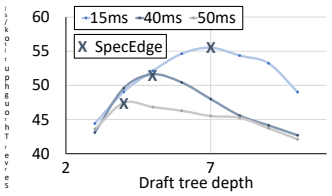


图 10：在不同网络时延下，服务器吞吐量随草稿深度的变化情况。

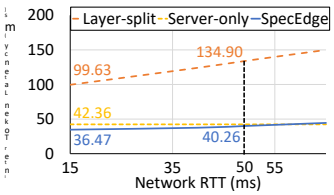


图 11：根据网络往返时延的 token 间时延。

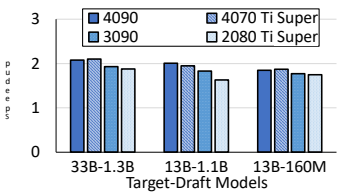


图 12：在自回归解码下，不同边缘设备上的 SpecEdge 加速增益。

SpecEdge 在服务器端吞吐量上的显著增加最终导致了更高的成本效率。这种吞吐量的提升由服务器端流水线感知调度（ §4.3 ）驱动，该调度交错处理多个请求，有效提高了服务器利用率，同时由主动式边缘草稿生成（ §5.2 ）驱动，增加了每次验证生成的平均 token 数量。我们还报告了每个验证周期生成的 token 的平均值和标准差。

除了吞吐量的提升外，图 6 显示，与仅服务器端推测解码相比，SpecEdge 在各种批量大小下平均将 token 间时延降低了 11.24%。这种双重收益 —— 更高的吞吐量和更低的时延 —— 对于资源效率和用户体验都至关重要的交互式大语言模型应用尤其有价值。

我们将评估扩展到其他模型，使用 Vicuna-33B 和 Llama2-13B-Chat 作为目标大语言模型（ LLM ），涵盖包括 C4 、 OAsst 、 WikiText-2 和 MTBench 在内的多种数据集。结果显示，主动式边缘草稿生成 SpecEdge 始终提高了性能，当草稿模型生成更深入的草稿并与目标 LLM 更好地对齐时，观察到的收益更大。这种对齐质量与服务器吞吐量的提升和 token 间时延的降低直接相关。所有模型和数据集组合的完整结果见附录 B.1。

5.2 逐组件效益

如图 7 所示，服务器 -only 基线将资源同时用于草稿生成和验证，导致每个草稿生成 - 验证周期服务器占用时间延长。相比之下，SpecEdge 将服务器专用于验证，同时将草稿生成任务卸载到成本更低的边缘设备，从而在所有模型配置中将服务器运行时间减少约 40-50%。

然而，单纯的解耦本身可能会增加时延并导致服务器未充分利用 (§ 4.1)。图 8 对比了三种渐进式实现方法，使用 14B/1.7B 模型组合：基本解耦、带主动式边缘草稿生成的解耦 (§ 4.2) 以及带流水线感知调度的完整 SpecEdge (§ 4.3)。基本解耦的吞吐量仅为 32.76 tokens/s，而且时延更高，而加入主动式草稿生成后，token 间时延降低至 28ms。带流水线感知调度的完整 SpecEdge 则显著将服务器吞吐量提高到 67.89 tok/s （ 2.07× 提升），同时保持时延不变，展示了各组件的互补优势。

每次验证生成的 token 数量。图 9 比较了三组目标 – 草稿模型之间每个验证周期的生成 token 数量，在仅服务器推测解码和 SpecEdge 之间的差异。平均而言，SpecEdge 每轮验证生成的 token 数量高出 13.21%。对于 32B–1.7B 配置，SpecEdge 每次验证生成 4.5 个 token，而仅服务器方法生成 4.02 个 token ； 而 14B–0.6B 配对则看到类似的增长（ 4.45 对 3.66 ）。这种效率提升源于

主动草稿树扩展，其中每个完整的草稿对齐都允许在后续轮次中生成更深的草稿树，从而显著提高验证效率和整体系统性能。

管线感知的草稿深度调整。我们展示了动态调整边缘草稿的前向传递次数（如 (§4.3) 所述），能够使草稿阶段与服务器验证及网络往返时延相匹配，从而在实践中实现最佳吞吐量。图 10 展示了在使用 32B/1.7B 模型对的情况下，不同 RTT 下服务器端吞吐量随草稿树深度的变化。平均而言，验证需要 94.2 毫秒，而每次草稿模型的前向传递大约需要 11 毫秒。当 RTT 为 15 毫秒时，SpecEdge 将草稿深度设置为七；RTT 为 40 毫秒时，将深度设置为五；而 RTT 为 50 毫秒时，则进一步降低到四。结果表明，SpecEdge 能够在不同网络条件下调整草稿深度以实现峰值吞吐量。

5.3 系统敏感性分析

网络 RTT 敏感性。我们在 SpecBench 上使用 Qwen32B，比较不同网络往返时延（ RTT ）下 SpecEdge 、按层切分配置和仅服务器配置的平均 token 间时延。按层切分配置采用自回归解码，其中四分之一模型层在边缘 RTX4090 上执行，其余模型层在服务器 A100 上执行。仅服务器配置完全在 A100 上运行树状推测解码，不受网络 RTT 影响。图 11 表明，在 RTT 低于 50 ms 时，SpecEdge 的 token 间时延低于仅服务器基线；RTT 为 15 ms 时降低 13.90% （ 36.47 ms 对比 42.36 ms ）。即使 RTT 达到 65 ms，SpecEdge 的时延也仅较 15 ms RTT 时增加 22.00% （达到 44.47 ms ），仍具有竞争力。相比之下，按层切分要慢得多：RTT 为 15 ms 时，其时延为 SpecEdge 的 2.73× （ 99.63 ms 对比 36.47 ms ）；RTT 为 50 ms 时进一步增加至 3.35× （ 134.90 ms 对比 40.26 ms ）。这种适应性来自 SpecEdge 更少的通信轮次与主动式边缘草稿生成，它们能够比按层切分方法更有效地抵消网络时延。

在不同边缘设备上的性能表现。图 12 展示了在服务器由不同边缘设备（ RTX 4090 、 4070 Ti Super 、 3090 和 2080 Ti ）协助的情况下，SpecEdge 在三种目标 - 草稿模型组合中实现的加速比。加速比是相对于仅使用 A100 服务器 GPU 进行的默认自回归解码测量的。所有模型组合中一致的加速比验证了该架构对不同硬件配置的鲁棒性。如预期，更强大的边缘 GPU 如 RTX 4090 提供了更大的加速效果，而即使是更经济的选择如 RTX 2080 Ti 仍然能提供显著的加速。这表明 SpecEdge 的方法在各种边缘硬件能力下仍然有效，允许部署具有灵活性。使用轻量级 GPU （ 3060 Ti 和 2080 Ti ）的更多结果可参见附录 B.2。

使用替代草稿生成方法的性能。 我们还评估了 SpecEdge 在非树 推测解码 方法下的表现 [Leviathan 等人， 2023] 以展示其超越树结构方法的多样性。使用这种替代架构，SpecEdge 在服务器吞吐量方面（ Llama2 13B/TL 1.1B 配对）提高了 1.96× ，在成本效率方面相比仅服务器部署提升了 1.67×。性能提升在不同模型组合中保持一致，即使是最小的 草稿模型 (JF 160M) 也实现了 1.52× 的吞吐量提升，同时保持终端用户的加速效果。完整结果请参见附录 B.3。

使用批量生成方法的性能。我们探索了一种替代的生成配置，其中单个边缘 GPU 通过批处理处理并发请求。这种方法使操作人员能够减少边缘 GPU 的数量。对各种批量大小的实验显示了更好的成本效率与时延增加之间的权衡。在预算有限且时延容忍度较高的部署中，这种配置可能具有优势。模型组合的完整结果见附录 B.4。

推理模式下的性能。 现代大语言模型提供推理模式以提升输出质量，其中推理 token 可能影响推测解码效率 [魏等 , 2023, 团队 , 2025]。为了探索 SpecEdge 在推理模式下的性能，我们测量了启用和未启用推理时的接受 token 数量、服务器吞吐量和成本效率。附录 B.5 的结果显示，当推理模式开启时，这三项指标均有一致提升，这意味着我们的系统从推理过程中的冗余中固有地受益，从而增强了推测解码性能。

使用各种 GPU 提供商的成本分析。 为了确保 SpecEdge 的成本效益发现能够超越单一提供商的适用性，我们在不同的云环境中验证了 SpecEdge 的性能。

我们比较了使用来自多个 GPU 提供商（ Vast.ai [Vas, 2025], Runpod [run,2025]，以及 TensorDock [Ten, 2025]）和云服务提供商（ Google Cloud Platform [GCP,2025], Amazon Web Services [AWS, 2025]，以及 Microsoft Azure [Azu, 2025]）的 GPU 结果，并考虑了定价差异。在所有测试的配置中，SpecEdge 一直提供成本效益的提升，确认我们的架构优势无论在何种特定云基础设施下都能持续发挥作用。详细的跨提供商比较见附录 C。

详细案例研究。为了补充我们的量化评估，附录 D 提供了 SpecEdge 实际运行的可视化展示。通过使用 Llama 模型响应有关戴森球的查询，我们追踪了完整的 token 生成生命周期 —— 从初始草稿到验证，以及随后的加速生成。该案例研究特别突出了两个关键的操作优势：（ 1 ）在服务器验证阶段，通过主动扩展，边缘 GPU 如何保持高生产力；（ 2 ）成功的草稿对齐如何在随后的轮次中产生更深入的草稿候选。

6 结论

我们提出了 SpecEdge，一种边缘辅助的 LLM 推理框架。它利用用户侧消费级 GPU 生成候选 token，服务器则专注于最终验证。通过仅传输已生成的输出，SpecEdge 可在典型广域网条件下高效运行。用户侧的主动式边缘草稿生成充分利用边缘 GPU，并降低端到端时延；服务器端的流水线感知验证调度则通过高效聚合和处理验证请求，确保较高吞吐量。实验结果表明，SpecEdge 通过实现 **2.22×** 的服务器吞吐量提升，显著降低运营成本，获得 **1.91×** 的成本效率改善，并比仅服务器基线略微降低时延。总体而言，SpecEdge 释放了边缘侧强大消费级 GPU 尚未利用的潜力，为未来 LLM 服务部署提供了一种可扩展且经济高效的方法。

更广泛的影响

这项工作通过将边缘端的初步 token 生成与服务器端的验证相结合，重新定义了大语言模型（ LLM ）服务中的劳动分工，超越了传统的集中式范式。这一范式转变不仅在不需要额外数据中心基础设施的情况下提高了服务器吞吐量，还为 LLM 服务提供了新的商业模式。通过利用边缘 GPU—— 无论是通过用户自有设备还是边缘云提供商 —— 我们的方法减少了对昂贵集中式服务器的依赖，使服务提供商能够以极低的成本提供可扩展、高性能的推理服务。这种去中心化的架构使企业能够将基础设施适应于边缘资源，从而解锁更灵活且具有成本效益的部署策略。此外，SpecEdge 缓解了与 … 相关的成本限制。 草稿生成，为推测解码研究开辟新途径。通过促进更丰富、更精确的草稿 token 生成方法的发展，它推动了大语言模型服务的性能和能力。

局限性与未来工作

SpecEdge 的设计考虑了灵活性，支持用户在边缘绘图阶段贡献自己 GPU 的场景。我们的成本效益分析主要集中在使用从边缘云提供商租用的消费级 GPU 部署的情况，但用户自有 GPU 可能进一步提高成本节约和可扩展性。将 SpecEdge 扩展以充分利用用户操作的硬件，为去中心化和社区驱动的推理带来了令人兴奋的机会。与此同时，当不受信任的设备参与计算时，这类场景在容错和安全等方面也带来了新的挑战。尽管 SpecEdge 已经支持分布式多用户环境，但探索这些更广泛的系统和安全方面是未来工作的重要方向。

致谢

我们感谢匿名审稿人提供的有益反馈和改进我们工作的建议。本工作得到韩国政府信息通信技术规划评估院（ IITP ）（ MSIT ）（编号 RS-2024-00398157 ）的支持。

参考文献

Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024.

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

Albert Q Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7b. *arXiv preprint arXiv:2310.06825*, 2023.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Nvidia. GeForce RTX 4090, 2025a. `https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090/`.

Nvidia. NVIDIA ADA GPU ARCHITECTURE, 2024a. `https://images.nvidia.com/aem-dam/Solutions/geforce/ada/nvidia-ada-gpu-architecture.pdf`.

Nvidia。NVIDIA A100，2025b。https://www.nvidia.com/en-us/data-center/a100/。

谷歌云。https://cloud.google.com/，2025。

Vast.ai，全球 GPU 市场。https://vast.ai/，2025 年。

Valve. Steam Hardware & Software Survey, 2024. `https://store.steampowered.com/hwsurvey/Steam-Hardware-Software-Survey-Welcome-to-Steam`.

Nvidia. NVIDIA Financial Reports, 2024b. `https://investor.nvidia.com/financial-info/financial-reports/default.aspx`.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 3505–3506, 2020.

Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. IEEE, 2022.

Nvidia. NVLink and NVLink Switch, 2025c. `https://www.nvidia.com/en-us/data-center/nvlink/`.

Nvidia. The NVIDIA Quantum InfiniBand Platform, 2025d. `https://www.nvidia.com/en-us/networking/products/infiniband/`.

英伟达。TensorRT-LLM, 2024c。https://github.com/NVIDIA/TensorRT-LLM。

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023.

Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning*, pages 19274–19286. PMLR, 2023.

Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.

Nikhil Bhendawade, Irina Belousova, Qichen Fu, Henry Mason, Mohammad Rastegari, and Mahyar Najibi. Speculative streaming: Fast llm inference without auxiliary models. *arXiv preprint arXiv:2402.11131*, 2024.

Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D Lee, Deming Chen, and Tri Dao. Medusa: Simple llm inference acceleration framework with multiple decoding heads. *arXiv preprint arXiv:2401.10774*, 2024.

Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. Eagle: Speculative sampling requires rethinking feature uncertainty. *arXiv preprint arXiv:2401.15077*, 2024.

Lawrence Stewart, Matthew Trager, Sujan Kumar Gonugondla, and Stefano Soatto. The n-grammys: Accelerating autoregressive inference with learning-free batched speculation. *arXiv preprint arXiv:2411.03786*, 2024.

Jun Zhang, Jue Wang, Huan Li, Lidan Shou, Ke Chen, Gang Chen, and Sharad Mehrotra. Draft & verify: Lossless large language model acceleration via self-speculative decoding. *arXiv preprint arXiv:2309.08168*, 2023.

Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, Xinhao Cheng, Zeyu Wang, Zhengxin Zhang, Rae Ying Yee Wong, Alan Zhu, Lijie Yang, Xiaoxiang Shi, et al. Specinfer: Accelerating large language model serving with tree-based speculative inference and verification. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 932–949, 2024.

Zhuoming Chen, Avner May, Ruslan Svirschevski, Yuhsun Huang, Max Ryabinin, Zhihao Jia, and Beidi Chen. Sequoia: Scalable, robust, and hardware-aware speculative decoding. *arXiv preprint arXiv:2402.12374*, 2024.

Ruslan Svirschevski, Avner May, Zhuoming Chen, Beidi Chen, Zhihao Jia, and Max Ryabinin. Specexec: Massively parallel speculative decoding for interactive llm inference on consumer devices. *arXiv preprint arXiv:2406.02532*, 2024.

Ziteng Sun, Ananda Theertha Suresh, Jae Hun Ro, Ahmad Beirami, Himanshu Jain, and Felix Yu. Spectr: Fast speculative decoding via optimal transport. *Advances in Neural Information Processing Systems*, 36, 2024.

Alexander Borzunov, Max Ryabinin, Artem Chumachenko, Dmitry Baranchuk, Tim Dettmers, Younes Belkada, Pavel Samygin, and Colin A Raffel. Distributed inference and fine-tuning of large language models over the internet. *Advances in Neural Information Processing Systems*, 36, 2024.

Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. Helix: Distributed serving of large language models via max-flow on heterogeneous gpus. *arXiv preprint arXiv:2406.01566*, 2024.

Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. {Zero-offload}: Democratizing {billion-scale} model training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pages 551–564, 2021.

Bharadwaj Pudipeddi, Maral Mesmakhosroshahi, Jinwen Xi, and Sujeeth Bharadwaj. Training large neural networks with constant memory using a new execution algorithm. *arXiv preprint arXiv:2002.05645*, 2020.

Nadav Timor, Jonathan Mamou, Daniel Korat, Moshe Berchansky, Oren Pereg, Moshe Wasserblat, Tomer Galanti, Michal Gordon-Kiwkowitz, and David Harel. Distributed speculative inference (dsi): Speculation parallelism for provably faster lossless language model inference. In *The Thirteenth International Conference on Learning Representations*, 2025.

Tianyu Liu, Yun Li, Qitan Lv, Kai Liu, Jianchen Zhu, Winston Hu, and Xiao Sun. Pearl: Parallel speculative decoding with adaptive draft length. In *The Thirteenth International Conference on Learning Representations*, 2025.

Bradley McDanel. Amusd: Asynchronous multi-device speculative decoding for llm acceleration. *arXiv preprint arXiv:2410.17375*, 2024.

MLC team. MLC-LLM, 2023-2024. URL `https://github.com/mlc-ai/mlc-llm`.

Yixin Song, Zeyu Mi, Haotong Xie, and Haibo Chen. Powerinfer: Fast large language model serving with a consumer-grade gpu. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 590–606, 2024.

Zhenliang Xue, Yixin Song, Zeyu Mi, Le Chen, Yubin Xia, and Haibo Chen. Powerinfer-2: Fast large language model inference on a smartphone. *arXiv preprint arXiv:2406.06282*, 2024.

Yiping Kang, Johann Hauswald, Cao Gao, Austin Rovinski, Trevor Mudge, Jason Mars, and Lingjia Tang. Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. *ACM SIGARCH Computer Architecture News*, 45(1):615–629, 2017.

Zhi Zhou, Xu Chen, En Li, Liekang Zeng, Ke Luo, and Junshan Zhang. Edge intelligence: Paving the last mile of artificial intelligence with edge computing. *Proceedings of the IEEE*, 107(8): 1738–1762, 2019.

Xiaofei Wang, Yiwen Han, Victor CM Leung, Dusit Niyato, Xueqiang Yan, and Xu Chen. Convergence of edge computing and deep learning: A comprehensive survey. *IEEE Communications Surveys & Tutorials*, 22(2):869–904, 2020.

A Vaswani. Attention is all you need. *Advances in Neural Information Processing Systems*, 2017.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.

J Redmon. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016.

Tausif Diwan, G Anirudh, and Jitendra V Tembhurne. Object detection using yolo: Challenges, architectural successors, datasets and applications. *multimedia Tools and Applications*, 82(6): 9243–9275, 2023.

Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. Efficiently scaling transformer inference. *Proceedings of Machine Learning and Systems*, 5:606–624, 2023.

Qwen Team. Qwen3, April 2025. URL `https://qwenlm.github.io/blog/qwen3/`.

Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E Gonzalez, et al. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, march 2023. *URL https://lmsys. org/blog/2023-03-30-vicuna*, 3 (5), 2023.

Mengzhou Xia, Tianyu Gao, Zhiyuan Zeng, and Danqi Chen. Sheared llama: Accelerating language model pre-training via structured pruning. *arXiv preprint arXiv:2310.06694*, 2023.

Peiyuan Zhang, Guangtao Zeng, Tianduo Wang, and Wei Lu. Tinyllama: An open-source small language model. *arXiv preprint arXiv:2401.02385*, 2024.

Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Findings of the Association for Computational Linguistics ACL 2024*, pages 7655–7671, Bangkok, Thailand and virtual meeting, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.findings-acl.456. URL https://aclanthology.org/2024.findings-acl.456.

Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.

Andreas Köpf, Yannic Kilcher, Dimitri von Rütte, Sotiris Anagnostidis, Zhi Rui Tam, Keith Stevens, Abdullah Barhoum, Duc Nguyen, Oliver Stanley, Richárd Nagyfi, et al. Openassistant conversations-democratizing large language model alignment. *Advances in Neural Information Processing Systems*, 36, 2024.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. *arXiv preprint arXiv:1609.07843*, 2016.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric Xing, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36:46595–46623, 2023.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models, 2023. URL https://arxiv.org/abs/2201.11903.

Runpod。https://runpod.io/，2025。

TensorDock — 简单且实惠的云 GPU。https://tensordock.com，2025。

亚马逊网页服务（AWS）。https://aws.amazon.com，2025。

微软 Azure。https://azure.microsoft.com，2025。

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

附加评估设置的详细信息

对于 SpecEdge 和仅服务器的推测解码基线，我们采用了来自近期基于树的推测解码的树构建算法 [Chen 等 , 2024, Svirschevski 等 , 2024]。在给定指定的树大小的情况下，每次草稿模型的前向传递会生成多个并行候选 token，然后根据累计对数概率进行剪枝，以使 token 的总数保持在树预算内。前向传递的总次数（草稿树深度）作为超参数设置。在我们的主要实验中，我们将每个请求的草稿树大小设置为 32。对于仅服务器基线，我们通过穷举搜索找到并选择最佳草稿树深度，而 SpecEdge 则根据第 4.3 节所述的验证和网络时延来确定其草稿深度。

在整个评估过程中，我们使用现成的目标大语言模型（ LLM ）和草稿模型，无需任何额外的微调。在主要实验中，我们使用 Qwen3 32B/14B 作为目标 LLM，Qwen3 1.7B/0.6B 作为草稿模型，并使用 SpecBench 数据集。为了展示更广泛的适用性，附录中包括了使用 Vicuna 33B 和 Llama2-13B-Chat-hf 作为目标 LLM，以及 Sheared Llama 1.3B 、 Tiny Llama 1.1B 和 JackFram 160M 作为草稿模型，在 C4 、 OAsst 、 WikiText-2 和 MTBench 上进行的实验。对于每个查询，我们生成最多 256 个输出 token。由于 SpecEdge 中的预推理阶段可以在服务器和边缘同时进行并行处理，因此预推理时延由 max(server_time,edge_time) 决定，这使得首 token 时延等指标与基线具有可比性；因此，我们报告的指标主要关注预推理后的输出 token。除非另有说明，实验中始终使用 SpecBench （涵盖六个不同任务）。

B 综合性能分析

B.1 在多样化模型和数据集上的性能

在表 2 中，我们展示了 SpecEdge 在不同目标 – 草稿模型对以及数据集上的适应性，使用批量大小为 1。我们测量了服务器吞吐量和成本效益提升，以及在 token 间时延上实现的加速。表 3 对 SpecEdge 和仅服务器推测解码基线相对于自回归解码参考的这些加速进行了比较。

我们的评估展示了 SpecEdge 在包括 Qwen 、 Vicuna 和 Llama 在内的多个模型上的有效性，这些模型的参数最多可达到 330 亿。虽然未来计划探索更大模型，但我们预期在扩展时无需进行根本性的设计更改。

Table2：SpecEdge 与仅服务器方法之间的吞吐量和成本效率比较。

		Gen. tokens per verify		Server Throughput (tok/s)		Cost Efficiency (1k toks/$)	
Target/Draft	Dataset	Server-only	**SpecEdge**	Server-only	**SpecEdge**	Server-only	**SpecEdge**
Vicuna 33B /SL 1.3B	C4	3.47±1.41	4.41±2.03	28.12	49.96 **(1.78x)**	24.99	38.23 **(1.53x)**
	OAsst	3.55±1.37	4.35±2.06	28.72	49.94 **(1.74x)**	25.53	37.89 **(1.48x)**
	WikiText-2	3.42±1.45	4.19±1.98	27.26	47.94 **(1.76x)**	24.23	36.42 **(1.50x)**
	MTBench	3.62±1.72	4.57±2.13	29.62	52.70 **(1.78x)**	26.33	40.21 **(1.53x)**
Llama2 13B /TL 1.1B	C4	3.40±1.35	3.48±1.34	36.80	70.76 **(1.92x)**	32.72	53.72 **(1.64x)**
	OAsst	3.57±1.40	3.67±1.39	38.55	69.49 **(1.80x)**	34.27	52.75 **(1.54x)**
	WikiText-2	3.52±1.37	3.61±1.40	37.69	70.21 **(1.86x)**	33.50	53.30 **(1.59x)**
	MTBench	3.67±1.43	3.75±1.46	40.96	76.69 **(1.87x)**	36.41	58.21 **(1.60x)**
Llama2 13B /JF 160M	C4	3.03±1.34	3.13±1.37	42.72	66.20 **(1.55x)**	37.98	50.36 **(1.33x)**
	OAsst	2.79±1.37	2.85±1.37	39.19	56.52 **(1.44x)**	34.84	42.95 **(1.23x)**
	WikiText-2	2.72±1.37	2.72±1.39	37.87	55.39 **(1.46x)**	33.66	42.08 **(1.25x)**
	MTBench	2.80±1.39	2.80±1.43	40.80	60.42 **(1.48x)**	36.42	45.92 **(1.26x)**

表 3：仅服务器、 SpecEdge 相对于自回归的加速情况。

Target/Draft Model	Dataset	ITL (ms)		Speedup	
		Server-only	**SpecEdge**	Server-only	**SpecEdge**
Vicuna 33B/SL 1.3B	C4	33.229	30.612	1.86x	2.02x
	OAsst	34.496	30.460	1.76x	2.00x
	WikiText-2	34.77	30.631	1.69x	1.92x
	MTBench	33.948	31.168	1.82x	1.98x
Llama2 13B/TL 1.1B	C4	23.687	21.851	1.63x	1.76x
	OAsst	24.661	22.377	1.54x	1.70x
	WikiText-2	23.883	21.338	1.59x	1.78x
	MTBench	23.136	21.463	1.66x	1.79x
Llama2 13B/JF 160M	C4	20.6	19.007	1.87x	2.03x
	OAsst	23.711	22.378	1.60x	1.70x
	WikiText-2	26.222	24.579	1.45x	1.54x
	MTBench	23.458	22.199	1.64x	1.73x

我们分割计算范式的核心原则 —— 将部分解码工作从服务器卸载到边缘 —— 通过 draft-verify 推测解码方案自然地扩展到任何规模的模型。我们即将进行的研究将量化这些好处在整个模型规模范围内的影响。

B.2 轻量级 GPU 的性能

我们使用 RTX 4090 作为消费级 GPU 的代表，这类 GPU 现在在边缘云提供商和用户设备中都很普遍。然而，为了展示更广泛的可推广性，我们进行了额外的实验，使用较轻的消费级 GPU。我们使用 RTX 3060 Ti 和 RTX 2080 Ti 测量了 SpecEdge 的性能。表 4 显示了在目标 / 草稿模型对配置为 Qwen3-14B/1.7B 和 Qwen3-14B/0.6B 时的 token 间时延和吞吐量。与仅服务器的方法相比，即使使用性能较低的 GPU，SpecEdge 仍能实现显著的吞吐量提升。

表 4：轻量级边缘 GPU 的性能比较。

Target/Draft Model	Edge GPU	Peak FP16 TFLOPS	Memory Bandwidth (GB/s)	Inter token Latency (ms)	Server Throughput (tok/s)
Qwen3-14B/1.7B	RTX 3060 Ti	16.20	448.0	36.818	50.297
	RTX 2080 Ti	26.90	616.0	34.409	54.657
	Server-only (A100 40GB)	312	1555	32.451	30.816
Qwen3-14B/0.6B	RTX 3060 Ti	16.20	448.0	36.818	56.135
	RTX 2080 Ti	26.90	616.0	34.409	54.657
	Server-only (A100 40GB)	312	1555	32.326	30.935

B.3 使用非基于树的推测解码方法的性能

为验证我们的方法在树状结构以外的适用性，我们实现了 [Leviathan 等人 2023]，提出的推测解码技术。该技术使用线性候选序列，而不探索多条分支路径。这使我们能够评估 SpecEdge 的核心创新，即将草稿生成与验证分离到边缘和服务器两侧，是否能够有效推广到不同的推测解码范式。

表 5 和表 6 展示了此次评估的结果，将 SpecEdge 与传统的仅服务器部署进行了比较。表 5 中的吞吐量测量结果展示了 SpecEdge 的效率

表 5：在 SpecEdge 上使用非树基推测解码方法的服务器吞吐量和成本分析。

	Server Throughput (tokens/s)		Cost Efficiency (1k tokens/$)	
Target/Draft Model	Server-only	**SpecEdge**	Server-only	**SpecEdge**
Vicuna 33B/SL 1.3B	24.527	39.370 (1.61x)	17.484	24.649 (1.41x)
Llama2 13B/TL 1.1B	33.044	64.851 (1.96x)	29.372	49.150 (1.67x)
Llama2 13B/JF 160M	32.755	49.639 (1.52x)	29.116	37.621 (1.29x)

表 6：使用非基于树的推测解码方法在 SpecEdge 上的 token 间时延 (ITL) 比较。

	ITL (ms)		Speedup	
Target/Draft Model	Server-only	**SpecEdge**	Server-only	**SpecEdge**
Vicuna 33B/SL 1.3B	40.771	38.067	1.61x	1.72x
Llama2 13B/TL 1.1B	30.263	26.085	1.40x	1.62x
Llama2 13B/JF 160M	30.530	30.818	1.38x	1.37x

优势，而表 6 量化了相对于仅服务器自回归解码的相对加速。这些结果确认了我们的分离架构无论底层推测解码策略如何，都能提供一致的改进，强化了我们方法的广泛适用性。

B.4 批处理草稿方法的性能

虽然我们的主要部署架构假设每个并发请求使用专用的边缘 GPU，但我们研究了一种替代架构，其中单个边缘 GPU 为多个并发请求生成草稿 token。这种替代方法为希望使用更少消费级边缘 GPU 的场景提供了折衷。我们通过在 RTX 4090 草稿机上对多个请求进行批处理，批量大小从 2 到 4，评估这种配置，同时测量不同目标 / 草稿模型对的 token 间时延和成本效率。

结果显示了成本效率与时延之间的权衡。如表 7 所示，由于更好的边缘 GPU 利用率，使用该替代方法的成本效率有所提高，在不同配置下提升幅度为 4.4% 到 29.5%。然而，这会带来 token 间时延增加的代价（慢 5.9% 到 19.0%），主要是由于批量处理多个请求的竞争以及较长的草稿到验证周期所致。在对成本敏感且可以接受更高时延的场景中，这种替代部署方法可能更可取。

表 7：替代部署方法的性能比较。

Target/Draft Model	Batch Size	Inter Token Latency (ms)	Cost Efficiency (1k toks/$)
Qwen3-14B/1.7B	2	31.777 (6.8% slower)	67.299 (14.7% better)
	3	36.617 (13.0% slower)	71.369 (29.5% better)
	4	40.099 (16.7% slower)	85.662 (24.5% better)
Qwen3-14B/0.6B	2	30.494 (8.5% slower)	67.680 (5.8% better)
	3	34.028 (15.2% slower)	80.252 (12.0% better)
	4	39.035 (19.0% slower)	82.376 (20.0% better)
Qwen3-32B/1.7B	2	43.539 (5.9% slower)	51.245 (4.4% better)
	3	48.889 (3.9% slower)	59.635 (21.3% better)
	4	58.402 (9.5% slower)	57.857 (25.1% better)

B.5 推理模式下的性能

许多现代大语言模型（ LLM ）支持推理能力，以提高其推断性能。推理 token 通常表现出高度重复的模式，这可能影响推测解码中的接受率。为了研究具有推理能力的推断性能，我们测量了启用和未启用推理模式时的接受 token 数量、服务器吞吐量和成本效率。

表 8 展示了每个目标 - 草稿模型对的接受 token 、服务器吞吐量和成本效率。在生成推理 token 时，我们观察到所有三个指标都有一致的提升：接受 token 、服务器吞吐量和成本效率。这些结果表明，SpecEdge 可以从推理过程中冗余带来的改进的推测解码性能中本质上受益。

表 8：SpecEdge 开启与关闭推理模式的性能比较。

	Accepted Tokens		Server Throughput (tok/s)		Cost Efficiency (1k toks/$)	
Target/Draft Model	Non-Reasoning	Reasoning	Non-Reasoning	Reasoning	Non-Reasoning	Reasoning
Qwen3-14B/1.7B	3.80 ± 1.54	4.17 ± 1.39	64.343	72.249	48.883	54.898
Qwen3-14B/0.6B	3.68 ± 1.50	4.10 ± 1.36	70.703	81.890	53.693	62.157
Qwen3-32B/1.7B	4.09 ± 1.95	4.62 ± 1.90	24.880	27.617	39.430	43.373

C 使用各种 GPU 提供商和云服务提供商的成本效益

我们将成本估算锚定在广泛可用的公开定价上：来自 Google Cloud Platform 的 A100 GPU 和来自 Vast.ai 的 RTX 4090 GPU [Vas, 2025]。为了验证我们关于成本效率的主张的稳健性，我们在多个云环境中进行了额外实验，包括各种 GPU 提供商（ Vast.ai, Runpod [run, 2025], 以及 TensorDock [Ten, 2025] ）和主要云服务提供商（ Google Cloud Platform [GCP, 2025], Amazon Web Services [AWS, 2025], 以及 Microsoft Azure [Azu, 2025] ）。表 9 和表 10 展示了不同提供商的 GPU 定价。表 11 显示 SpecEdge 在所有测试配置中始终保持成本效率的提升。

表 9：边缘 GPU 定价。

GPU	GPU Provider	Cost ($/hr)
RTX 4090	RunPod	0.69
	Vast.ai	0.35
	TensorDock	0.359
RTX Pro 6000	Vast.ai	1.08
	TensorDock	1.15

表 10：服务器 GPU 定价。

GPU	Cloud Service Provider	Cost ($/hr)
A100 40GB	Google Cloud Platform	4.05
	Amazon Web Services	4.10
A100 80GB	Google Cloud Platform	5.05
	Amazon Web Services	5.12
	Microsoft Azure	3.673

D 案例研究：主动式边缘草稿生成实践

本节通过带有示例查询的说明性示例说明了 SpecEdge 的主动草稿生成机制。我们演示了系统如何处理完整的草稿生成生命周期：初始

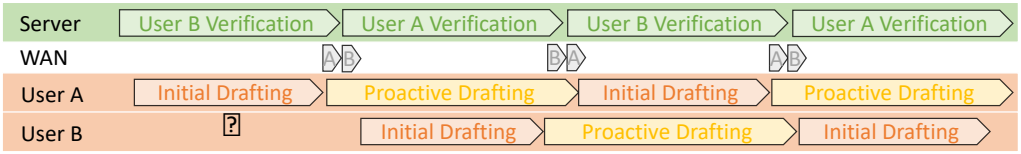


图 13：时间轴视图显示了边缘和服务器的并行操作，同时在服务器验证期间发生主动草稿生成。

表 11：GPU 提供商与云服务提供商的成本效率比较。

Target/Draft Model	Server/Edge GPU	GPU Provider	Cost Efficiency (1k toks/$)		
			Google Cloud Platform	Amazon Web Service	Microsoft Azure
Qwen3-14B/1.7B	A100 40GB / RTX 4090	Vast.ai	51.018 (1.87×)	50.485 (1.87×)	-
		RunPod	44.721 (1.64×)	44.311 (1.65×)	-
		TensorDock	50.829 (1.87×)	50.30(1.87×)	-
		Baseline (Server-only)	27.222	26.890	-
Qwen3-14B/1.7B	A100 40GB / RTX Pro 6000	Vast.ai	47.68 (1.75×)	47.30 (1.76×)	-
		TensorDock	46.64 (1.71×)	46.27 (1.72×)	-
		Baseline (Server-only)	27.222	26.890	-
Qwen3-14B/0.6B	A100 40GB / RTX 4090	Vast.ai	52.938 (1.86×)	52.386 (1.88×)	-
		RunPod	46.382 (1.63×)	45.957 (1.65×)	-
		TensorDock	52.741 (1.86×)	52.192 (1.88×)	-
		Baseline (Server-only)	28.415	27.802	-
Qwen3-32B/1.7B	A100 80GB / RTX 4090	Vast.ai	31.619 (1.82×)	31.332 (1.83×)	41.757 (1.75×)
		RunPod	28.453 (1.64×)	28.144 (1.65×)	36.280 (1.52×)
		TensorDock	31.619 (1.82×)	31.239 (1.83×)	41.591 (1.75×)
		Baseline (Server-only)	17.330	17.090	23.822

草稿生成、主动扩展、服务器验证以及随后的草稿生成。图 13 显示了每个操作的时间线视图。

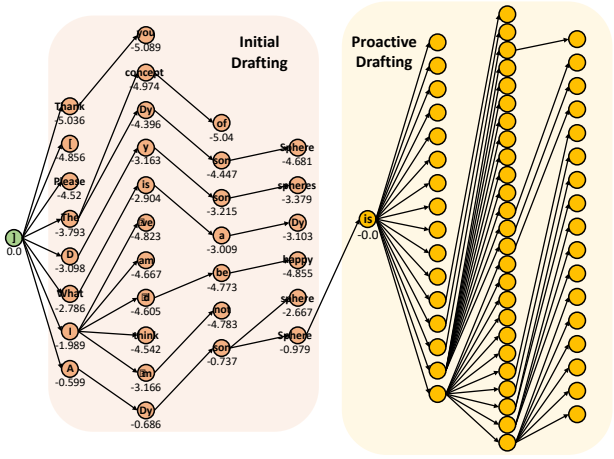


图 14：初始草稿生成后的主动草稿树扩展示例。在创建初始草稿树（左）后，边缘设备识别最可能的叶子 token，并在等待服务器验证的同时主动生成额外的 token （右）。

实验设置。在本演示中，我们使用来自 OAsst 数据集的一个样本，查询为 “ 戴森球是什么？ ”。我们使用 Llama-3.2-3B [Grattafiori 等人，2024] 作为目标模型，并使用 Llama-3.2-1B 作为草稿模型。

初稿与主动扩展。图 14 展示了从初稿草稿生成到主动扩展的过渡。一旦边缘设备构建了初稿树，它会将该树提交给服务器进行验证。与此同时 —— 而不是保持空闲 —— 边缘设备会识别具有最高累计对数概率的扩展节点，并开始主动生成额外的 token。

服务器验证与对齐。当来自服务器的验证结果到达时，边缘设备将其本地扩展的树与这些结果进行比较。图 15 展示了一个完整草稿对齐的案例，其中服务器验证的 token （ "A Dyson Sphere is" ）与初始草稿树路径及所选扩展节点均匹配。

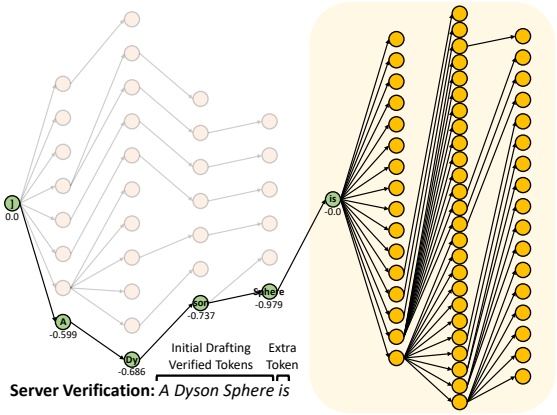


图 15：服务器验证和完整草稿对齐。服务器验证结果（显示 “ 一个戴森球是 ” ）与草稿树中的路径完美对齐，验证了初始草稿和所选择的扩展节点。

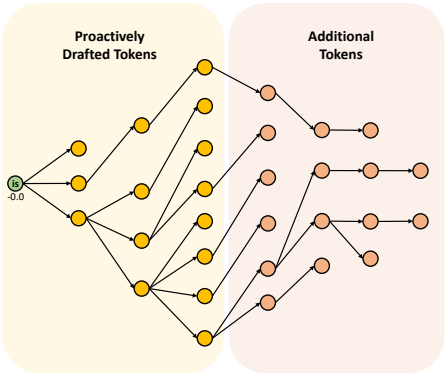


图 16：通过主动生成的 token 进行更深入的 token 草稿生成。之前生成的沿着额外 token （ “is” ）的主动 token 树可以在下一轮草稿生成中立即使用，而无需额外计算。

加速 token 草稿生成。在成功验证之后，SpecEdge 利用主动草稿生成的 token 来提高下一个草稿生成周期的效率。图 16 显示了这些预生成的 token 如何有助于下一次草稿提交，从而无需重新生成这些 token，并因此构建更深的草稿候选项。

性能影响。 此示例演示了 SpecEdge 的前瞻性草稿生成策略如何提供切实的性能收益：

- **减少空闲时间：**在服务器验证期间，边缘 GPU 仍保持高效工作。
- **更深入的后续草稿生成：**预生成的 token 允许在下一个草稿生成周期中进行额外的草稿前传。
- **有效的资源利用：**边缘和服务器上的计算资源都得到了最大化利用。

这里显示的完全对齐案例代表了最优情形，尽管 SpecEdge 同样处理部分或未对齐的草稿（第 4.2 节）。

NeurIPS 论文清单

1. **索赔**
   问题：摘要和引言中提出的主要论点是否准确反映了论文的贡献和范围?

   回答：[是]
   理由：摘要和引言包括论文的主要主张和贡献。

   指南：
   - 答案 NA 表示摘要和引言中未包含论文中提出的主张。
   - 摘要和 / 或引言应清楚地说明所提出的主张，包括论文中的贡献以及重要的假设和限制。对此问题回答 “ 否 ” 或 “ 不适用 ” 可能不会受到评审者的好评。
   - 所提出的主张应与理论和实验结果相符，并反映出这些结果在多大程度上可以推广到其他环境。
   - 只要明确这些目标并未通过论文实现，包含作为激励的愿望目标是可以的。

2. **局限性**
   问题：论文是否讨论了作者所进行工作的局限性?
   回答：[是]
   理由：限制在附录的限制部分中讨论。
   指南：
   - 答案 NA 意味着该论文没有限制，而答案 No 意味着该论文有一些限制，但这些限制未在论文中讨论。
   - 鼓励作者在他们的论文中创建一个单独的 “ 局限性 ” 部分。
   - 论文应指出任何强假设以及结果对这些假设违规的稳健性（例如，独立性假设、无噪声环境、模型规范性、渐近近似仅在局部成立）。作者应思考这些假设在实际中可能被违反的情况以及其可能的影响。
   - 作者应该反思所提出的主张的范围，例如，该方法是否仅在少数数据集或少数运行中进行了测试。一般来说，经验结果通常依赖于隐含假设，这些假设应该被明确说明。
   - 作者应当反思影响该方法性能的因素。例如，当图像分辨率低或在光线较暗的情况下拍摄时，面部识别算法的表现可能较差。或者，语音转文本系统可能无法可靠地为在线讲座提供字幕，因为它无法处理专业术语。
   - 作者应讨论所提出算法的计算效率以及它们随数据集规模的扩展情况。
   - 如适用，作者应讨论其方法在解决隐私和公平问题方面可能的局限性。
   - 尽管作者可能担心对局限性的完全诚实可能会被审稿人作为拒稿的理由，但更糟糕的情况可能是审稿人发现论文中未承认的局限性。作者应使用他们的最佳判断，并认识到为透明度采取的个人行动在制定维护社区诚信的规范中起着重要作用。审稿人将被特别指示不要因对局限性的诚实而进行惩罚。

3. **理论假设与证明**
   问题：对于每个理论结果，论文是否提供了完整的假设集合以及完整（且正确）的证明?

回答：[无]
理由：该论文不包含需要正式证明的定理或引理。公式编号合理，并清楚地说明了其符号。

指南：
- 答案 NA 意味着该论文不包含理论结果。
- 论文中的所有定理、公式和证明都应编号并进行交叉引用。
- 所有假设都应在任何定理的陈述中明确说明或引用。
- 证明可以出现在主论文中，也可以出现在补充材料中，但如果它们出现在补充材料中，建议作者提供一个简短的证明草图以提供直观理解。
- 反过来，论文正文中的任何非形式化证明，都应在附录或补充材料中提供相应的形式化证明。
- 证明所依赖的定理和引理应当被适当引用。

4. **实验结果的可重复性**

问题：论文是否充分披露了再现论文主要实验结果所需的所有信息，以至于影响论文的主要论点和 / 或结论（无论是否提供代码和数据）？

回答：[是]
理由说明：端到端系统和方法已被详细描述。用于评估的模型、数据集和超参数均提供了详细信息。

指南：
- 答案 NA 意味着论文中没有包括实验。
- 如果论文包含实验，对于这个问题回答 “ 不 ”，评审可能不会有好的印象：使论文具有可重复性很重要，无论是否提供代码和数据。
- 如果贡献是数据集和 / 或模型，作者应描述为使其结果可重现或可验证所采取的步骤。
- 根据贡献的不同，再现性可以通过各种方式实现。例如，如果贡献是一个新颖的架构，完整描述该架构可能就足够了；或者如果贡献是一个特定模型和实证评估，可能需要使他人能够使用相同的数据集复现该模型，或者提供对该模型的访问。一般来说，发布代码和数据通常是一种实现这一点的好方法，但再现性也可以通过提供详细的复现结果的操作指南、对托管模型的访问（例如，在大语言模型的情况下）、发布模型检查点，或其他适合所进行研究的方式来实现。
- 虽然 NeurIPS 不要求公开代码，但要求所有投稿提供合理的复现途径，具体方式可取决于贡献的性质。例如：(a) 若贡献主要是一种新算法，论文应说明如何复现该算法。(b) 若贡献主要是一种新模型架构，论文应清晰、完整地描述其架构。(c) 若贡献是一种新模型（例如大语言模型），应提供访问该模型以复现结果的方法，或提供复现模型的方法，例如开源数据集或构建数据集的说明。(d) 我们认识到某些情况下复现可能较为困难，作者可以说明其提供的具体复现方式。对于闭源模型，模型访问可能受到某些限制，例如仅面向注册用户，但其他研究人员仍应有途径复现或验证结果。

5. **数据和代码的开放获取**

问题：论文是否提供数据和代码的开放获取，并附有足够的说明，以便如补充材料所述，忠实地重现主要实验结果?

回答：[是]

理由说明：我们提交我们的代码作为补充材料，完全可复现。审稿后，我们将公开提交的代码。

指南：

- 答案 NA 意味着论文不包括需要代码的实验。
- 请参阅 NeurIPS 的代码和数据提交指南（ https://nips.cc/public/guides/CodeSubmissionPolicy ）以获取更多详细信息。
- 虽然我们鼓励发布代码和数据，但我们理解这可能不可行，因此 “ 不 ” 是可以接受的答案。仅仅因为未包含代码而拒稿是不允许的，除非代码是贡献的核心（例如，对于一个新的开源基准）。
- 说明应包含执行命令和运行环境的具体信息，以重现结果。有关更多详细信息，请参见 NeurIPS 代码和数据提交指南（ https://nips.cc/public/guides/CodeSubmissionPolicy ）。
- 作者应提供数据访问和准备的说明，包括如何访问原始数据、预处理数据、中间数据和生成的数据等。
- 作者应提供用于重现新提出方法和基线的所有实验结果的脚本。如果只能重现部分实验，他们应说明哪些实验未包括在脚本中以及原因。
- 投稿时，为保持匿名，作者应发布匿名化版本（如适用）。
- 建议在论文附带的补充材料中尽可能提供充分信息，也允许包含指向数据和代码的 URL。

6. **实验设置 / 细节**

问题：论文是否具体说明了理解结果所需的所有训练和测试细节（例如，数据拆分、超参数、选择方式、优化器类型等）？

回答：[是]

理由：所有实验设置和方法都已详细说明。

指南：

- 答案 NA 意味着论文中没有包括实验。
- 实验设置应在论文的核心部分呈现，详细程度应足以让人理解结果并理解其意义。
- 完整的详细信息可以通过代码、附录或作为补充材料提供。

7. **实验统计学显著性**

问题：论文是否适当地报告了误差条，并正确定义了或提供了关于实验统计显著性的其他适当信息?

回答：[是]

理由说明：误差线在评估图表和表格中提供。

指南：

- 答案 NA 意味着论文中没有包括实验。
- 如果结果附有误差线、置信区间或统计显著性检验（至少对于支持论文主要论点的实验），作者应回答 “ 是 ”。
- 应明确说明误差条所反映的变异因素，例如训练与测试集划分、初始化、某个参数的随机抽取，或在给定实验条件下整次运行的变化。

- 计算误差条的方法应予以说明（闭合形式公式、调用库函数、自助法等）。
- 应说明所作的假设（例如，误差呈正态分布）。
- 应该明确误差线是标准偏差还是均值的标准误。
- 报告 1- 标准差误差条是可以的，但应当声明清楚。如果误差的正态性假设未被验证，作者最好报告 2- 标准差误差条，而不是仅说明他们有 96% 的置信区间。
- 对于非对称分布，作者应注意不要在表格或图中显示对称的误差线，因为这可能会导致结果超出范围（例如，负错误率）。
- 如果在表格或图表中报告了误差线，作者应在正文中解释其计算方法，并在正文中引用相应的图表或表格。

8. **实验计算资源**

   问题：对于每个实验，论文是否提供了足够的信息，包括所需的计算资源（计算工作节点类型、内存、执行时间），以便重复实验?

   回答：[是]

   理由：我们指定每个实验设置使用的计算资源类型。

   指南：
   - 答案 NA 意味着论文中没有包括实验。
   - 论文应说明计算工作者的类型（ CPU 或 GPU ）、内部集群或云提供商，包括相关的内存和存储。
   - 论文应提供每个单独实验运行所需的计算量，并估算总计算量。
   - 论文应披露整个研究项目是否比论文中报告的实验需要更多的计算资源（例如，未列入论文的初步实验或失败实验）。

9. **道德规范**

   问题：论文中进行的研究是否在各方面都遵守 NeurIPS 伦理规范 https://neurips.cc/public/EthicsGuidelines?

   回答：[是]

   理由：论文中进行的研究在各方面都符合 NeurIPS 道德规范。

   指南：
   - 答案 NA 意味着作者尚未审核 NeurIPS 伦理守则。
   - 如果作者回答 “ 否 ”，他们应解释需要偏离《道德守则》的特殊情况。
   - 作者应确保保持匿名（例如，如果由于其管辖区的法律或法规有特殊考虑时）。

10. **更广泛的影响**

    问题：论文是否讨论了所进行工作的潜在积极社会影响和消极社会影响?

    回答：[是]

    理由：潜在的社会影响在附录的更广泛影响部分中讨论。

    指南：
    - 答案 NA 意味着所进行的工作没有社会影响。
    - 如果作者回答 “NA” 或 “ 否 ”，他们应解释为什么他们的工作没有社会影响，或为什么论文没有涉及社会影响。

- 负面社会影响的例子包括潜在的恶意或非预期用途（例如，虚假信息、生成虚假资料、监控）、公平性考虑（例如，部署可能对特定群体造成不公平影响的决策技术）、隐私考量以及安全考量。
- 会议预计，许多论文将是基础研究，并不与特定应用相关，更不用说部署了。然而，如果存在通向任何负面应用的直接路径，作者应指出。例如，指出生成模型质量的提升可能被用于生成用于虚假信息的深度伪造是合理的。另一方面，没有必要指出，通用的神经网络优化算法可能使人们更快地训练生成深度伪造的模型。
- 作者应当考虑在技术按预期使用并正常运行时可能产生的危害，技术按预期使用但给出错误结果时可能产生的危害，以及因技术的（故意或无意）误用而产生的危害。
- 如果存在负面的社会影响，作者还可以讨论可能的缓解策略（例如，限制模型的发布、在提供攻击的同时提供防御、监控滥用的机制、监控系统如何随着时间学习反馈的机制、提高机器学习的效率和可访问性）。

11. **保障措施**

问题：论文是否描述了为负责任地发布具有高误用风险的数据或模型（例如，预训练语言模型、图像生成器或抓取的数据集）而采取的保障措施?

回答 : [无]

理由：本文提出了一种新的服务系统范式，该范式与数据或模型的发布是正交的。

指南：

- 答案 NA 意味着该论文不存在此类风险。
- 具有高滥用风险或双重用途风险的发布模型应在必要的防护措施下发布，以允许对模型进行受控使用，例如要求用户遵守使用指南或访问模型的限制，或实施安全过滤。
- 从互联网上抓取的数据集可能存在安全风险。作者应描述他们如何避免发布不安全的图像。
- 我们认识到提供有效的保护措施是具有挑战性的，许多论文并不要求这样做，但我们鼓励作者考虑这一点，并尽最大努力做到尽职尽责。

12. **现有资产的许可证**

问题：论文中使用的资产（例如代码、数据、模型）的创建者或原始所有者是否得到了适当的认可，以及许可证和使用条款是否明确提及并得到适当遵守?

回答：[是]

理由：我们使用或复现的模型、数据和代码均已适当引用。

指南：

- 答案 NA 意味着该论文未使用现有资产。
- 作者应引用最初发布该代码包或数据集的论文。
- 作者应说明所使用的资产版本，并在可能的情况下提供网址。
- 每个资源都应包含许可证名称（例如，CC-BY 4.0 ）。
- 对于来自特定来源（例如网站）的抓取数据，应提供该来源的版权和服务条款。

- 如果资产被发布，包中的许可、版权信息和使用条款应当提供。对于流行的数据集，paperswithcode.com/datasets 为一些数据集整理了许可信息。他们的许可指南可以帮助确定数据集的许可。
- 对于已经重新打包的现有数据集，应提供原始许可证以及衍生资产的许可证（如果已更改）。
- 如果这些信息在网上不可获得，建议作者联系该资产的创作者。

13. **新资产**

问题：论文中引入的新资产是否有良好的文档记录，并且这些文档是否与资产一起提供?

回答：[是]

理由：论文中的端到端系统代码作为匿名压缩文件通过补充材料提交。代码附带有运行说明的文档。

指南：

- 答案 NA 意味着该论文未发布新资产。
- 研究人员应通过结构化模板在提交时传达数据集 / 代码 / 模型的详细信息。这包括训练、许可、限制等方面的细节。
- 论文应讨论是否以及如何从其资产被使用的人那里获得了同意。
- 在提交时，请记得将您的资料匿名化（如适用）。您可以创建一个匿名化的链接，或包含一个匿名化的压缩文件。

14. **众包与人体研究**

问题：对于众包实验和涉及人类受试者的研究，论文是否包括给参与者的完整指令文本和截图（如适用），以及关于补偿（如有）的详细信息?

回答 : [无]

理由：该论文不涉及众包，也不涉及人类受试者的研究。

指南：

- 答案 NA 意味着该论文不涉及众包，也不涉及人类受试者的研究。
- 将这些信息包含在补充材料中是可以的，但如果论文的主要贡献涉及人体受试者，那么应尽可能在正文中包含尽可能多的细节。
- 根据 NeurIPS 伦理规范，参与数据收集、整理或其他劳动的工作人员的报酬应至少达到数据收集者所在国家的最低工资标准。

15. **涉及人类受试者的研究的机构审查委员会（ IRB ）批准或同等审批**

问题：该论文是否描述了研究参与者可能面临的风险，这些风险是否已向受试者披露，以及是否获得了机构审查委员会（ IRB ）批准（或根据您所在国家或机构的要求获得了同等的批准 / 审查）？

回答 : [无]

理由：该论文不涉及众包，也不涉及人类受试者的研究。

指南：

- 答案 NA 意味着该论文不涉及众包，也不涉及人类受试者的研究。

- 根据研究进行的国家，任何涉及人类受试者的研究可能需要获得伦理审查委员会（ IRB ）批准（或同等批准）。如果您获得了 IRB 批准，您应在论文中明确说明这一点。
- 我们认识到，这方面的程序在不同的机构和地点之间可能有显著差异，我们希望作者遵守 NeurIPS 道德准则以及其所在机构的指导方针。
- 对于初次提交，请勿包含任何可能破坏匿名的信息（如适用），例如进行评审的机构。

16. **LLM 使用声明**

问题：如果论文中使用大语言模型（ LLM ）是该研究核心方法的重要、原创或非标准组成部分，论文是否描述了其使用情况？请注意，如果 LLM 仅用于写作、编辑或格式化目的，并且不影响核心方法、科学严谨性或研究原创性，则不需要声明。

回答 : [无]

理由：本研究中的核心方法开发并不涉及将大语言模型（ LLMs ）作为任何重要、原创或非标准的组成部分。

指南：

- 答案 NA 意味着本研究中的核心方法开发不涉及将大语言模型（ LLM ）作为任何重要的、原创的或非标准的组成部分。
- 请参阅我们的 LLM 政策（ https://neurips.cc/Conferences/2025/LLM ）了解应或不应描述的内容。